

Below is a **menu-driven C program for Binary Tree operations**. It includes creation, insertion, deletion, searching, all major traversals, height, node count, and leaf count.

C

```
#include <stdio.h>
#include <stdlib.h>

// Structure of a Binary Tree Node
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

// Create a new node
struct Node* createNode(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Insert a node using level-order insertion
struct Node* insert(struct Node *root, int value)
{
    struct Node *newNode = createNode(value);

    if (root == NULL)
        return newNode;

    struct Node *queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        struct Node *temp = queue[front++];

        // Insert in left position
```

```

        if (temp->left == NULL) {
            temp->left = newNode;
            return root;
        }
        else {
            queue[rear++] = temp->left;
        }

        // Insert in right position
        if (temp->right == NULL) {
            temp->right = newNode;
            return root;
        }
        else {
            queue[rear++] = temp->right;
        }
    }

    return root;
}

// Preorder: Root Left Right
void preorder(struct Node *root)
{
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Inorder: Left Root Right
void inorder(struct Node *root)
{
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Postorder: Left Right Root
void postorder(struct Node *root)
{
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// Level-order traversal

```

```

void levelOrder(struct Node *root)
{
    if (root == NULL) {
        printf("Tree is empty.\n");
        return;
    }

    struct Node *queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        struct Node *temp = queue[front++];

        printf("%d ", temp->data);

        if (temp->left != NULL)
            queue[rear++] = temp->left;

        if (temp->right != NULL)
            queue[rear++] = temp->right;
    }
}

// Search a node
int search(struct Node *root, int key)
{
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    if (search(root->left, key))
        return 1;

    return search(root->right, key);
}

// Count total nodes
int countNodes(struct Node *root)
{
    if (root == NULL)
        return 0;

    return 1 + countNodes(root->left)
        + countNodes(root->right);
}

// Count leaf nodes
int countLeafNodes(struct Node *root)

```

```

{
    if (root == NULL)
        return 0;

    if (root->left == NULL && root->right == NULL)
        return 1;

    return countLeafNodes(root->left)
        + countLeafNodes(root->right);
}

// Find height of tree
int height(struct Node *root)
{
    if (root == NULL)
        return -1;

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    if (leftHeight > rightHeight)
        return leftHeight + 1;
    else
        return rightHeight + 1;
}

// Find deepest/rightmost node and its parent
struct Node* getDeepestNode(struct Node *root, struct Node **parent)
{
    struct Node *queue[100];
    int front = 0, rear = 0;

    struct Node *temp = NULL;

    queue[rear++] = root;
    *parent = NULL;

    while (front < rear) {
        temp = queue[front++];

        if (temp->left != NULL) {
            *parent = temp;
            queue[rear++] = temp->left;
        }

        if (temp->right != NULL) {
            *parent = temp;
            queue[rear++] = temp->right;
        }
    }

    return temp;
}

```

```

}

// Delete a node from a binary tree
struct Node* deleteNode(struct Node *root, int key)
{
    if (root == NULL) {
        printf("Tree is empty.\n");
        return root;
    }

    // If root is the only node
    if (root->left == NULL && root->right == NULL) {

        if (root->data == key) {
            free(root);
            return NULL;
        }

        printf("Node not found.\n");
        return root;
    }

    struct Node *queue[100];
    int front = 0, rear = 0;

    struct Node *keyNode = NULL;
    struct Node *temp = NULL;
    struct Node *parent = NULL;

    queue[rear++] = root;

    // Find the node to delete
    while (front < rear) {

        temp = queue[front++];

        if (temp->data == key)
            keyNode = temp;

        if (temp->left != NULL) {
            parent = temp;
            queue[rear++] = temp->left;
        }

        if (temp->right != NULL) {
            parent = temp;
            queue[rear++] = temp->right;
        }
    }

    if (keyNode == NULL) {
        printf("Node not found.\n");
    }
}

```

```

        return root;
    }

    // temp is the deepest node
    struct Node *deepestParent = NULL;
    struct Node *deepest = getDeepestNode(root, &deepestParent);

    // Replace key node's data
    keyNode->data = deepest->data;

    // Delete deepest node
    if (deepestParent->right == deepest)
        deepestParent->right = NULL;
    else
        deepestParent->left = NULL;

    free(deepest);

    return root;
}

// Display all traversals
void displayTraversals(struct Node *root)
{
    if (root == NULL) {
        printf("Tree is empty.\n");
        return;
    }

    printf("\nPreorder  : ");
    preorder(root);

    printf("\nInorder   : ");
    inorder(root);

    printf("\nPostorder  : ");
    postorder(root);

    printf("\nLevelorder: ");
    levelOrder(root);

    printf("\n\n");
}

// Main function
int main()
{
    struct Node *root = NULL;

    int choice, value;

    while (1) {

```

```

printf("\n=====\\n");
printf("      BINARY TREE OPERATIONS\\n");
printf("=====\\n");
printf("1. Insert Node\\n");
printf("2. Delete Node\\n");
printf("3. Search Node\\n");
printf("4. Preorder Traversal\\n");
printf("5. Inorder Traversal\\n");
printf("6. Postorder Traversal\\n");
printf("7. Level-order Traversal\\n");
printf("8. Display All Traversals\\n");
printf("9. Count Total Nodes\\n");
printf("10. Count Leaf Nodes\\n");
printf("11. Find Height\\n");
printf("12. Exit\\n");
printf("=====\\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {

    case 1:
        printf("Enter value: ");
        scanf("%d", &value);

        root = insert(root, value);

        printf("Node %d inserted successfully.\\n", value);
        break;

    case 2:
        printf("Enter value to delete: ");
        scanf("%d", &value);

        root = deleteNode(root, value);
        break;

    case 3:
        printf("Enter value to search: ");
        scanf("%d", &value);

        if (search(root, value))
            printf("%d found in the tree.\\n", value);
        else
            printf("%d not found in the tree.\\n", value);

        break;

    case 4:
        printf("Preorder: ");

```

```

        preorder(root);
        printf("\n");
        break;

    case 5:
        printf("Inorder: ");
        inorder(root);
        printf("\n");
        break;

    case 6:
        printf("Postorder: ");
        postorder(root);
        printf("\n");
        break;

    case 7:
        printf("Level-order: ");
        levelOrder(root);
        printf("\n");
        break;

    case 8:
        displayTraversals(root);
        break;

    case 9:
        printf("Total nodes = %d\n", countNodes(root));
        break;

    case 10:
        printf("Leaf nodes = %d\n", countLeafNodes(root));
        break;

    case 11:
        printf("Height = %d\n", height(root));
        break;

    case 12:
        printf("Program terminated.\n");
        exit(0);

    default:
        printf("Invalid choice!\n");
}
}

```

Example

If you insert:

10 20 30 40 50 60 70

The tree becomes:

```
      10
     /  \
    20   30
   / \  / \
  40 50 60 70
```

The traversals will be:

```
Preorder   : 10 20 40 50 30 60 70
Inorder    : 40 20 50 10 60 30 70
Postorder  : 40 50 20 60 70 30 10
Levelorder : 10 20 30 40 50 60 70
```

Note: This is a **general binary tree**, not a Binary Search Tree (BST). Therefore, insertion is done level-by-level, and inorder traversal is **not necessarily sorted**.